

Transition: DS + Algorithms

Heaps



Reading: Probably too long!
(Extended next one to Friday...)
↳ over Hashing

Worksheet:



• Priority queues: An abstract data structure

Operations: Given priority queue PQ:

• $\text{insert}(e, k)$: adds e to PQ with ^{priority} k

• $\text{get Max}()$: returns max k in PQ.
maximum *or Min (if min queue)*

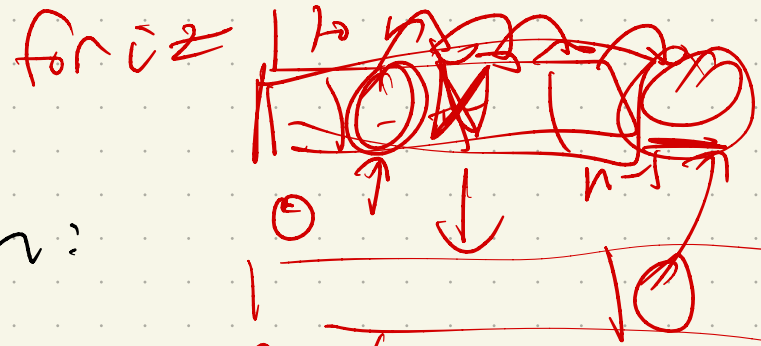
• $\text{remove Max}()$: removes max element
(plus size & empty) *(or min)*

*Note: no $\text{find}(k)$
or $\text{find}(e)$*

Why would this be useful?

*Min: fastest or shortest route
wait queue $\rightarrow$ airplanes*

How to implement?



Array or list option:

↳ keep elements in array

sorted order:

getMax: $O(1)$

insert: $O(n)$

removeMax: $O(1)$

unsorted order:

getMax: $O(n)$

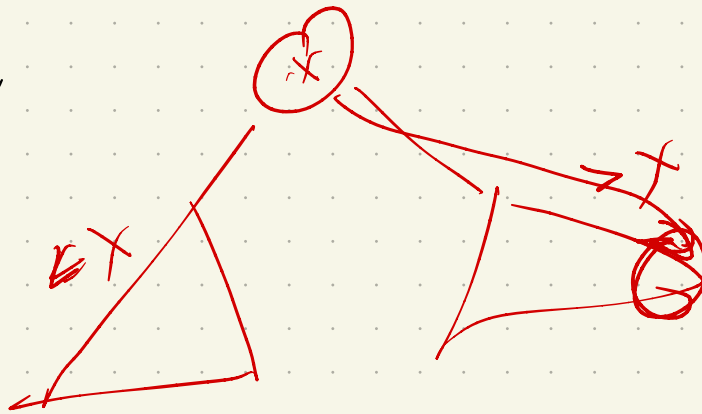
insert: $O(1)$
amortized

removeMax: $O(n)$

With a ~~BST~~:

AVL

remove &
insert: $O(\log n)$



$O(\text{height})$
 $= O(\log n)$

Doing it faster:

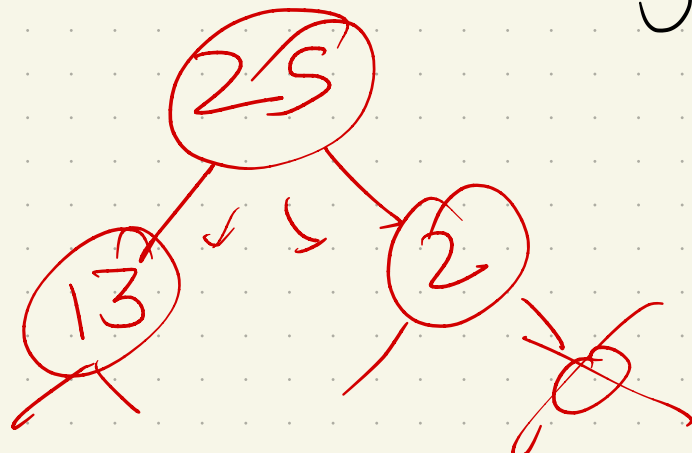
Heap: A binary tree where:

① For every node v (other than r) the key stored at v is $\leq$ the key stored at v 's parent

② The tree is complete: (array-based)

levels $0 \dots h-1$ are full & h is filled in left to right.

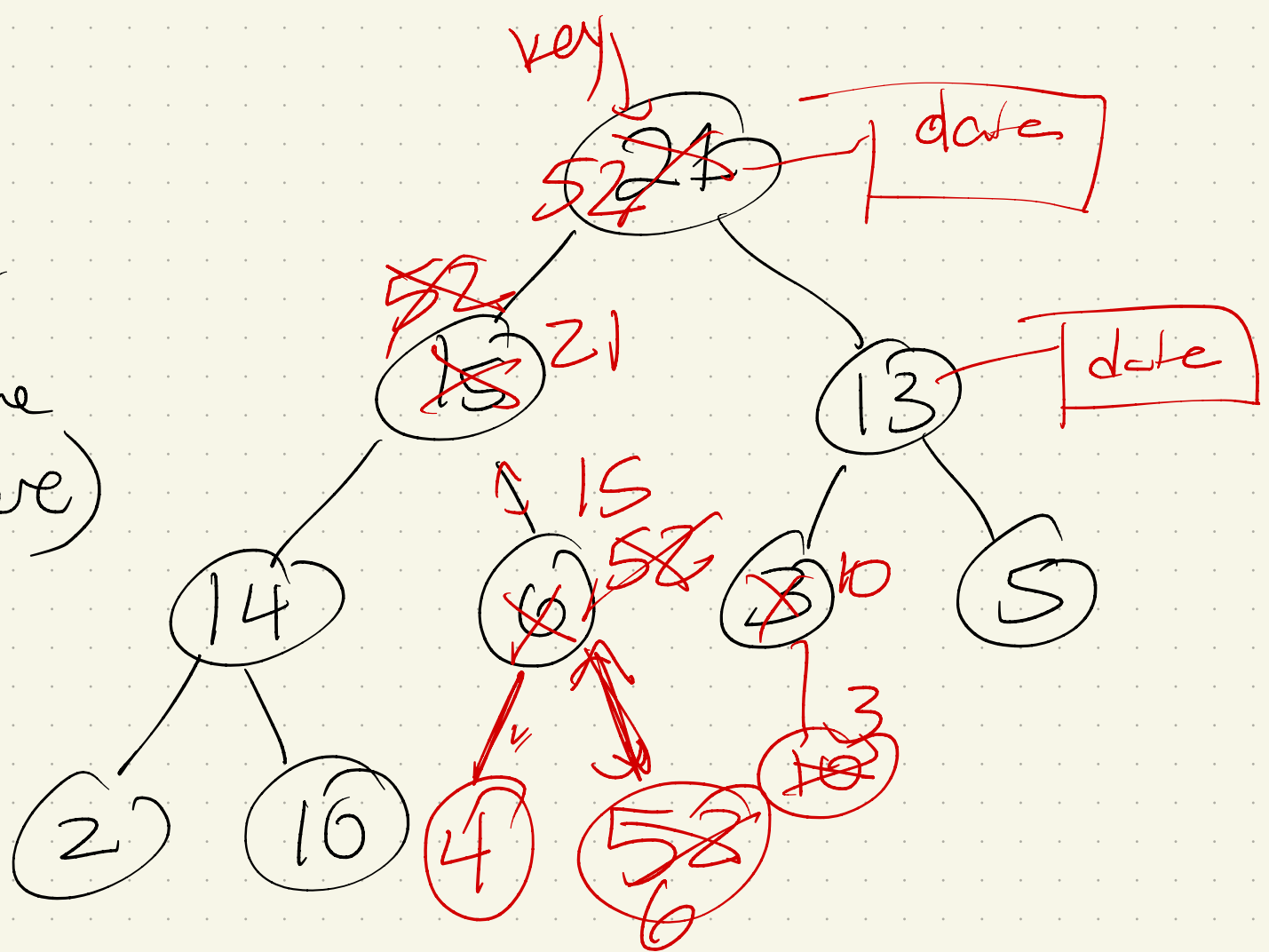
Ex:



perfect $\lceil \log_2 n \rceil$
+ no ptrs

Ex:

(Note: only showing the keys here)



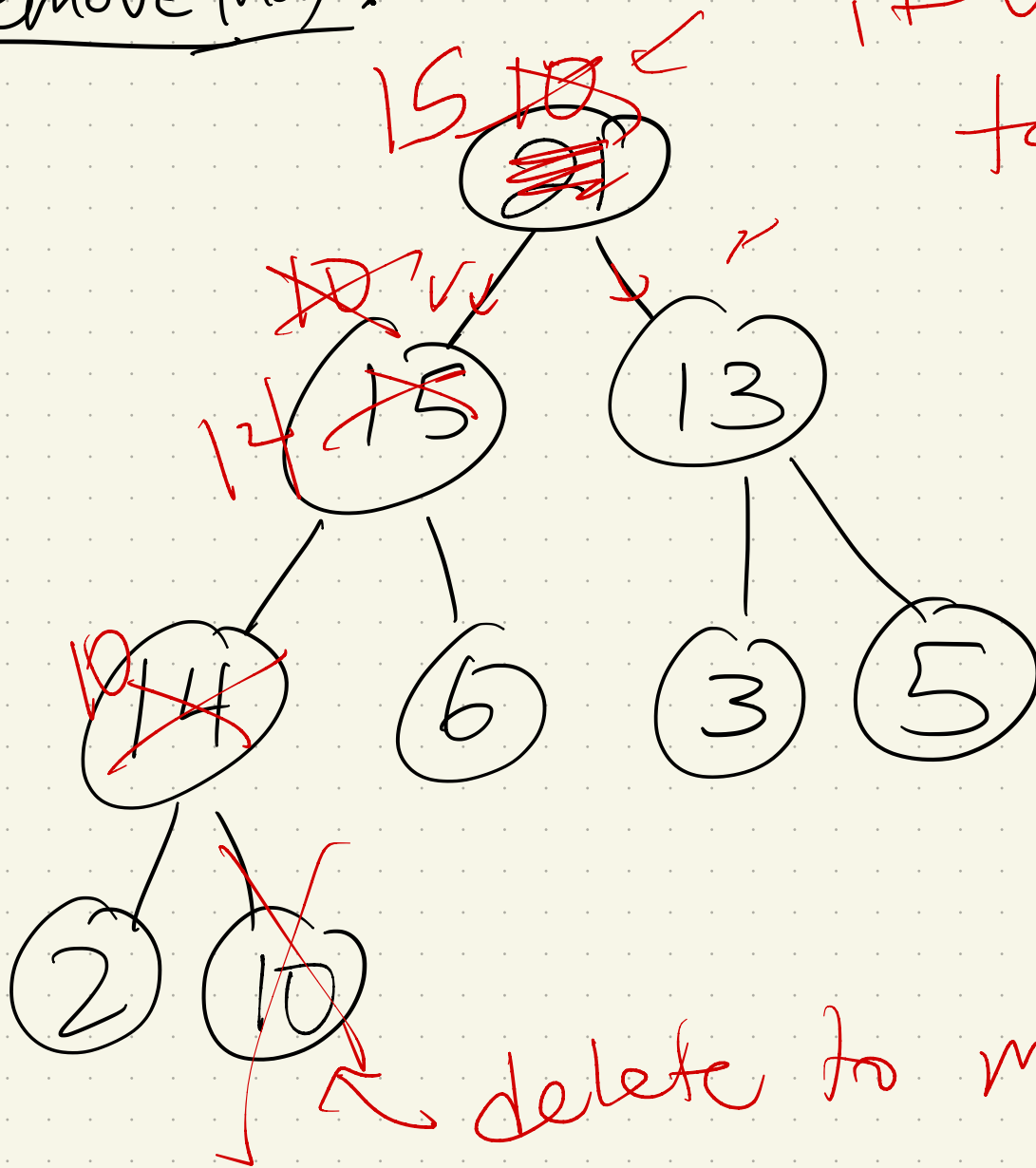
Insert(4)

Insert(52)

Insert(10)

Remove Max:

if unhappy
take larger child
& swap



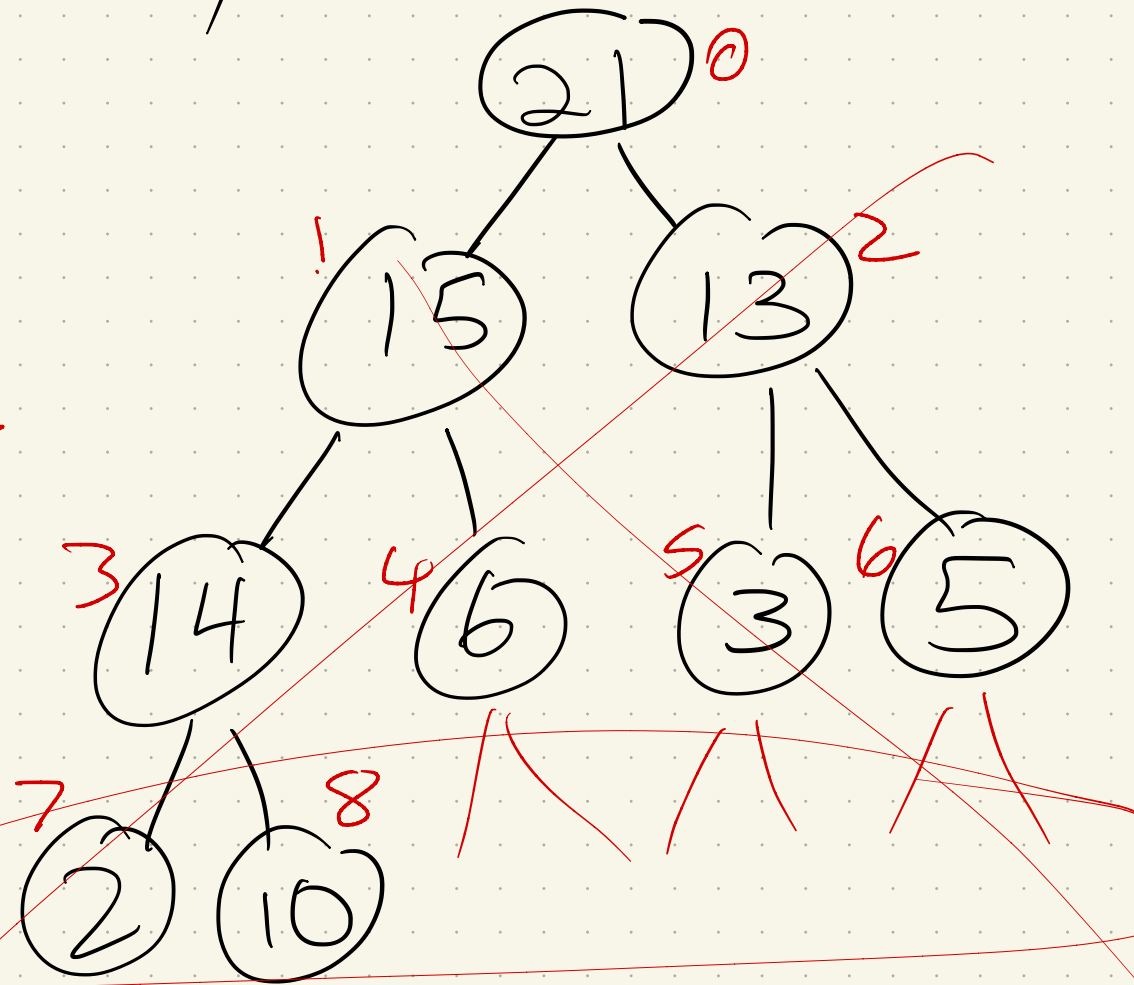
Implementation: Array based trees:

recall:

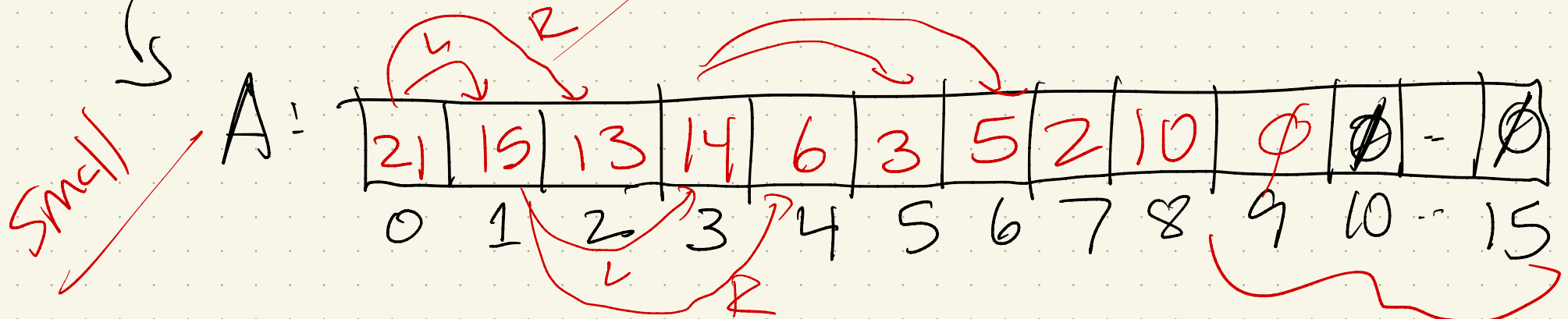
$$\text{left}(v) = 2v + 1$$

$$\text{right}(v) = 2v + 2$$

$$\text{parent}(v) = \lfloor \frac{v-1}{2} \rfloor$$



array



Runtime:

get Max: $O(1)$

Insert & remove Max:

$\leq \lceil \log_2 n \rceil$ comparisons
& swaps

$\rightarrow O(\log_2 n)$

Heap sort:

Main idea: Insert n items
into heap, then remove
them

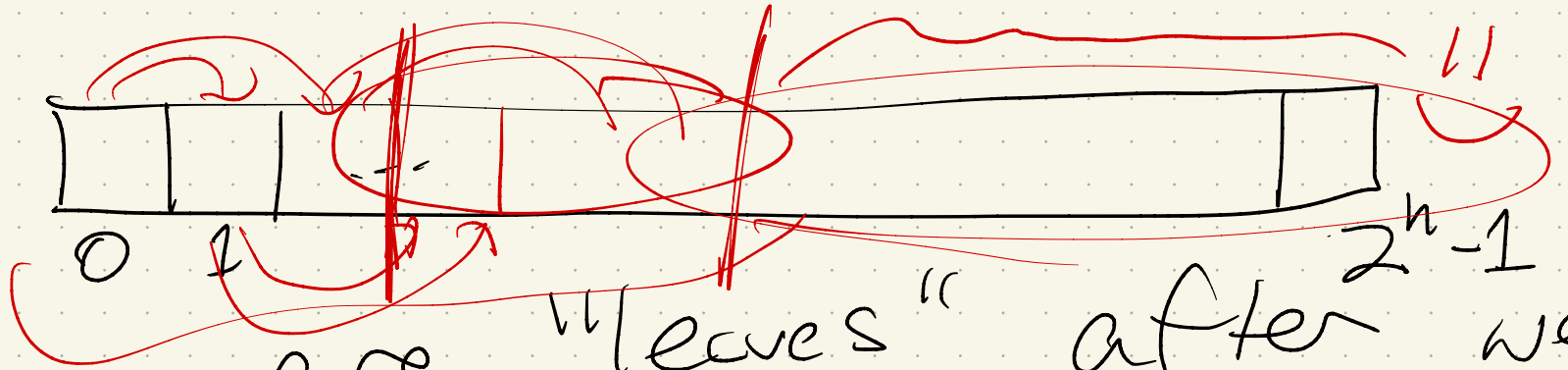
$$\begin{aligned} \hookrightarrow n \cdot \log_2 n + n \cdot \log_2 n \\ = O(n \log n) \end{aligned}$$

Practically: Can go faster!

$\hookrightarrow O(n)$ in practice

Unsorted array

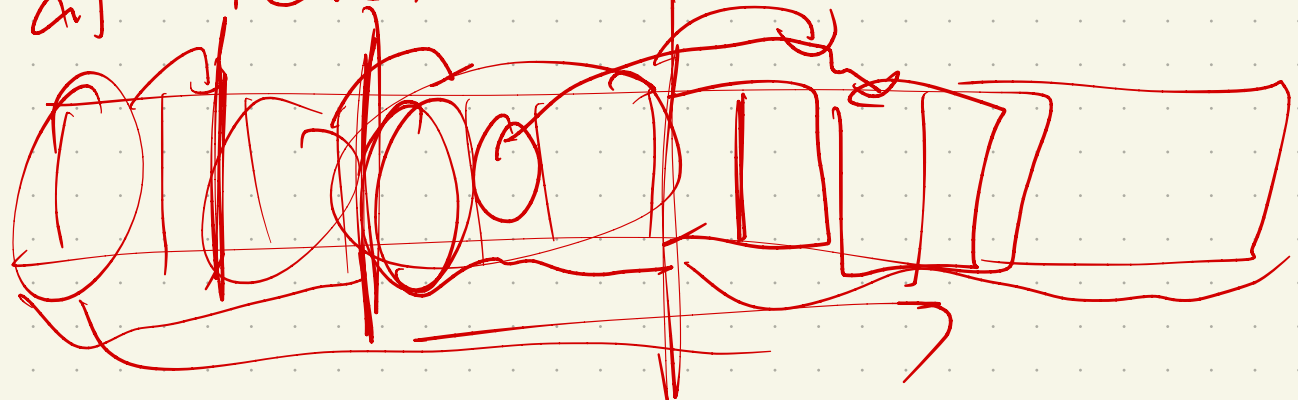
→ convert to heap



Where are "leaves" after we heapify? Back half: 2^{n-1} to $2^n - 1$

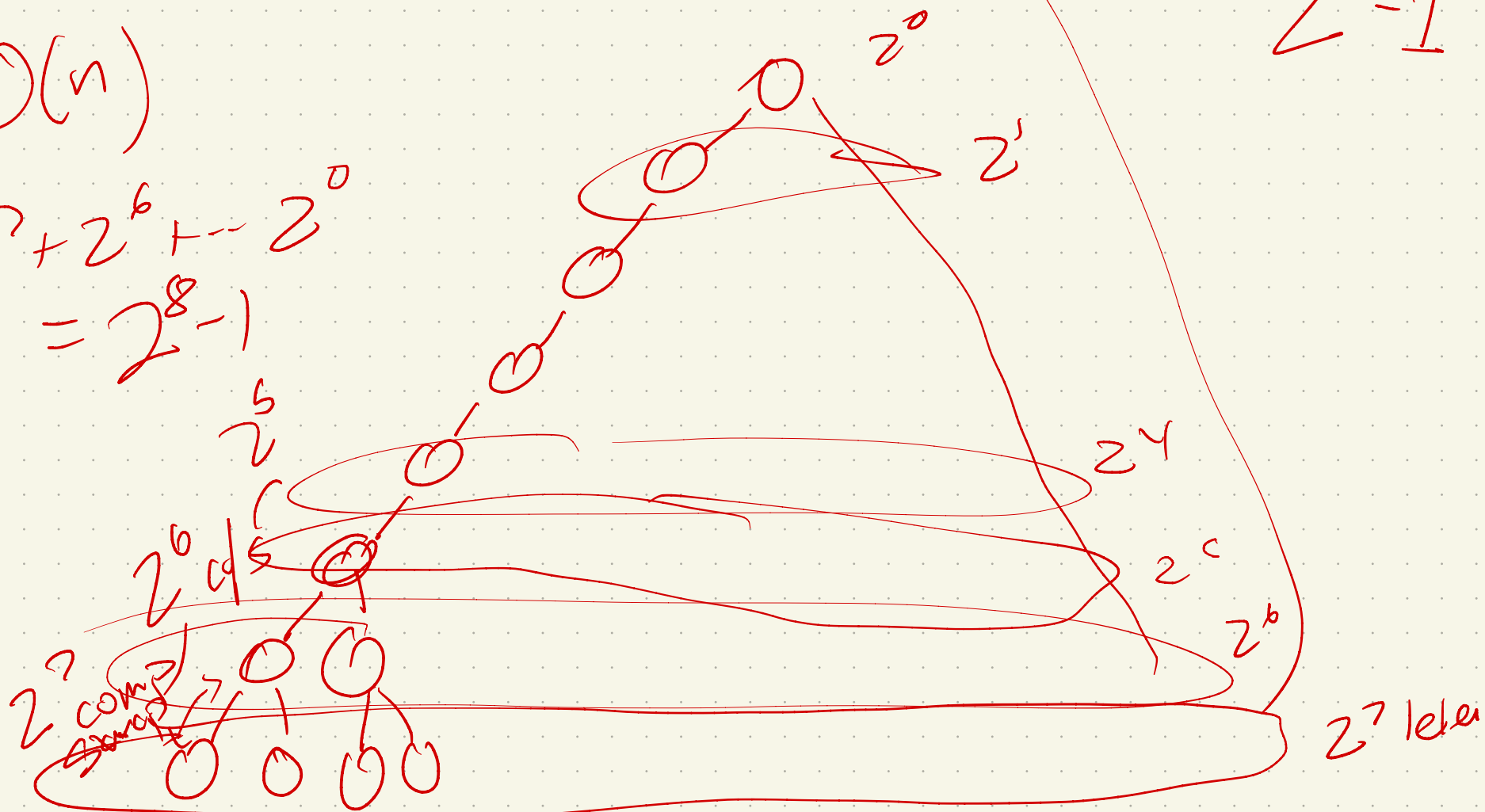
So: heapify: build bottom up

look at level $h-1$:



$$O(n)$$

$$2^7 + 2^6 + \dots + 2^0 = 2^8 - 1$$



Pictorially:

(see prev)

Adding operations:

Often a heap also supports:

Runtime: $\log_2 n$

changekey
decreaseKey

$\log_2 n$

locally swap

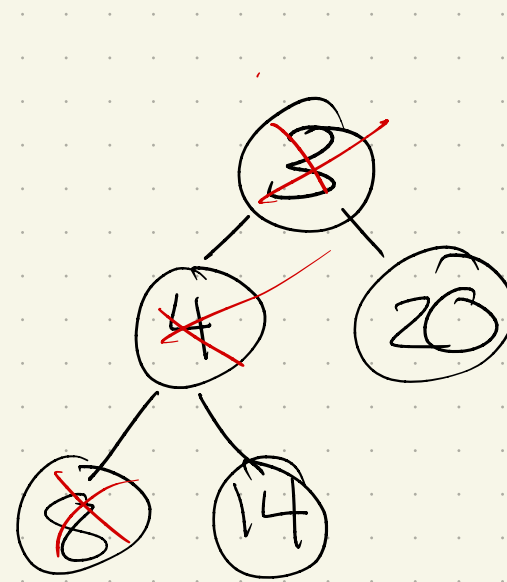
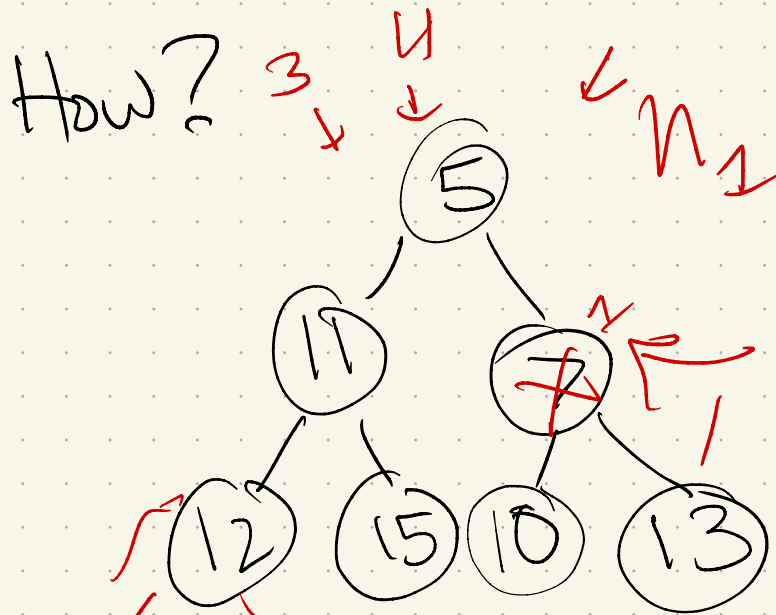
Note that this also gives us delete:

vary by library

if $Ern = 1$
pri = lo
other data

Another: Merge (H_1, H_2): ~~✱~~

Create a new heap with all values
of H_1 & H_2



$$n = n_1 + n_2$$

Runtime:
1. remove Min
↳ insert into H_1

~~$$n_1 \log n_1 +$$~~

~~$$n \log n$$~~

1
"Erin"

hidden data

4
"Kelly"

Binomial Heap

Goal: Improve Merge

$$O(n) \rightarrow O(\log n)$$

at the "cost" of min

$$O(1) \rightarrow O(\log n)$$

{But really not!!}

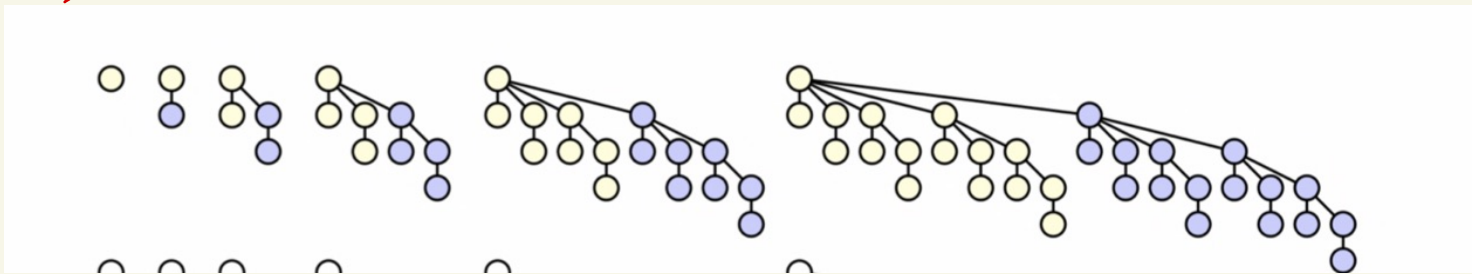
Amortization:

Dfn: A binomial tree, defined recursively:

Base case: B_0

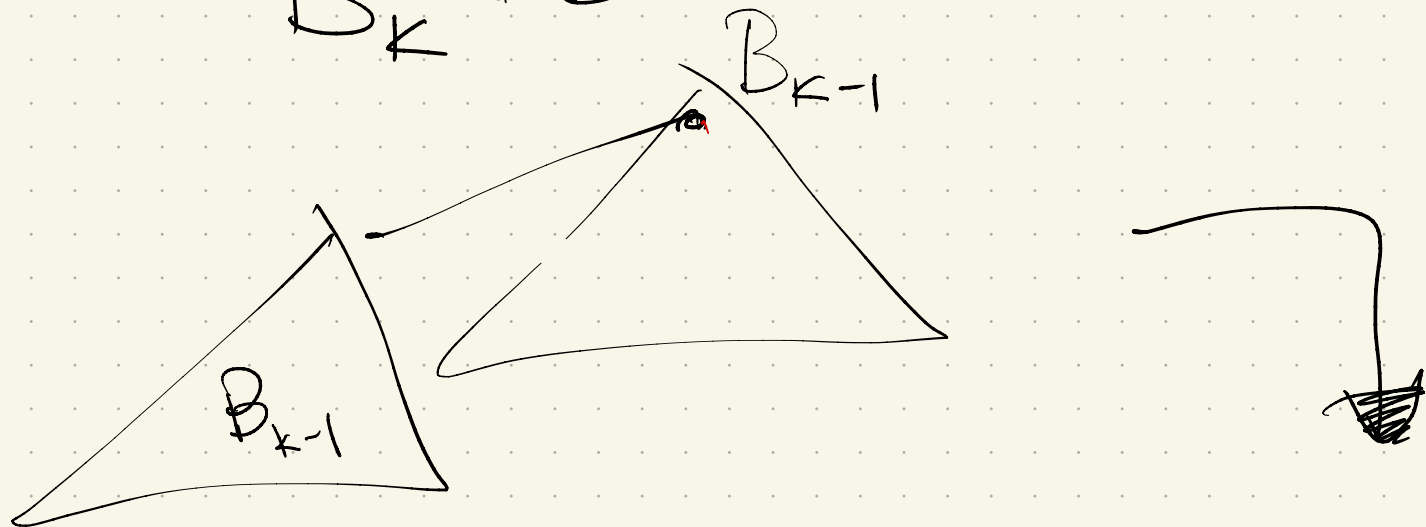
B_i : two copies of B_{i-1} , one root connected as (new) child of the other

Picture:

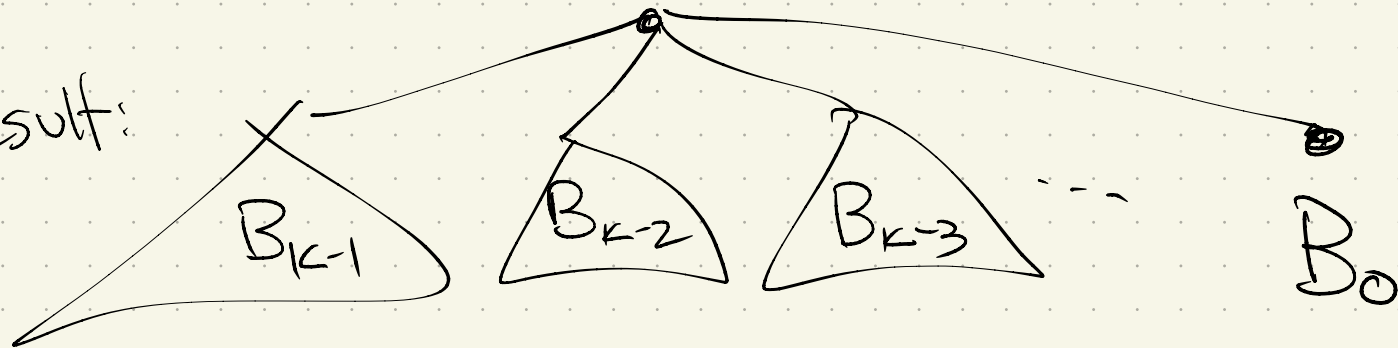


General pattern:

B_k is binomial tree of order k



End result:



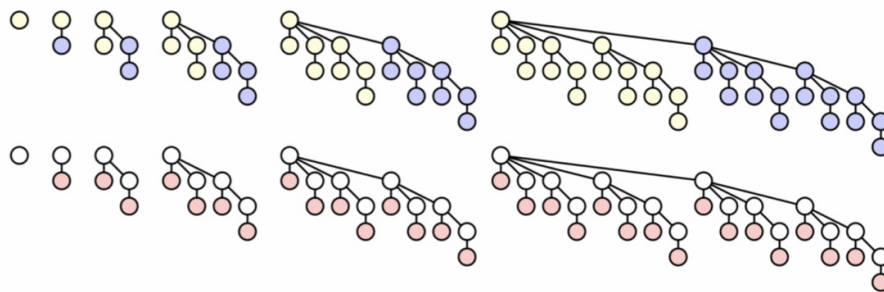
HWO question from previous semester!

4. A binomial tree of order k is defined recursively as follows:

- A binomial tree of order 0 is a single node.
- For all $k > 0$, a binomial tree of order k consists of two binomial trees of order $k - 1$, with the root of one tree connected as a new child of the root of the other. (See the figure below.)

Prove the following claims:

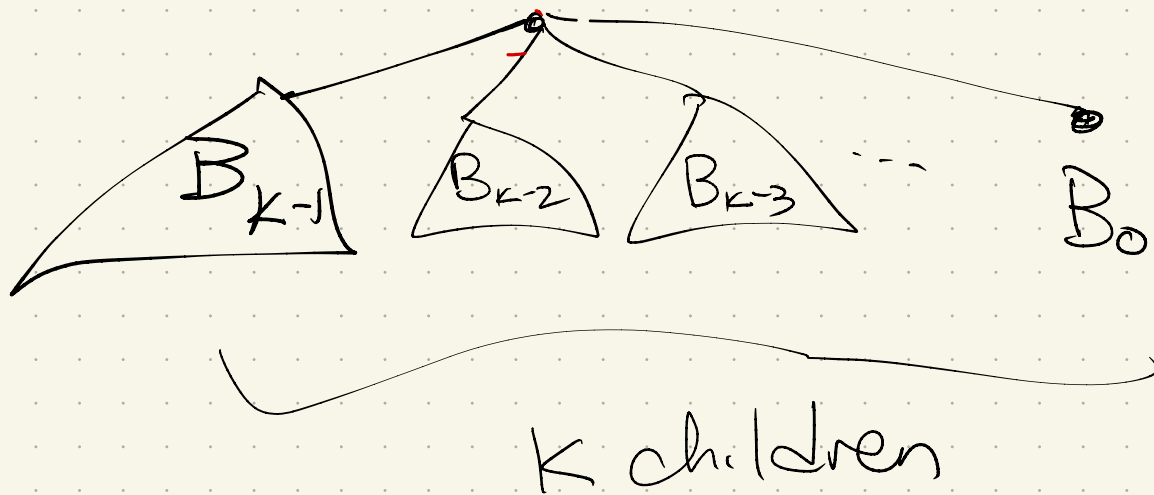
- For all non-negative integers k , a binomial tree of order k has exactly 2^k nodes.
- For all positive integers k , attaching a leaf to every node in a binomial tree of order $k - 1$ results in a binomial tree of order k .
- For all non-negative integers k and d , a binomial tree of order k has exactly $\binom{k}{d}$ nodes with depth d .



Binomial trees of order 0 through 5.
Top row: the recursive definition. Bottom row: the property claimed in part (b).

so size =

Height of $B(k)$:



which is biggest?

so $H(k) =$

in terms of # of nodes:
 B_k has 2^k nodes & height k
 $\Rightarrow$

Binomial Heap

- Like regular heap, child $>$ parent
(for all nodes)
- But: this is a collection of binomial trees,
with at most 1 of each size

so: one B_0

one B_1

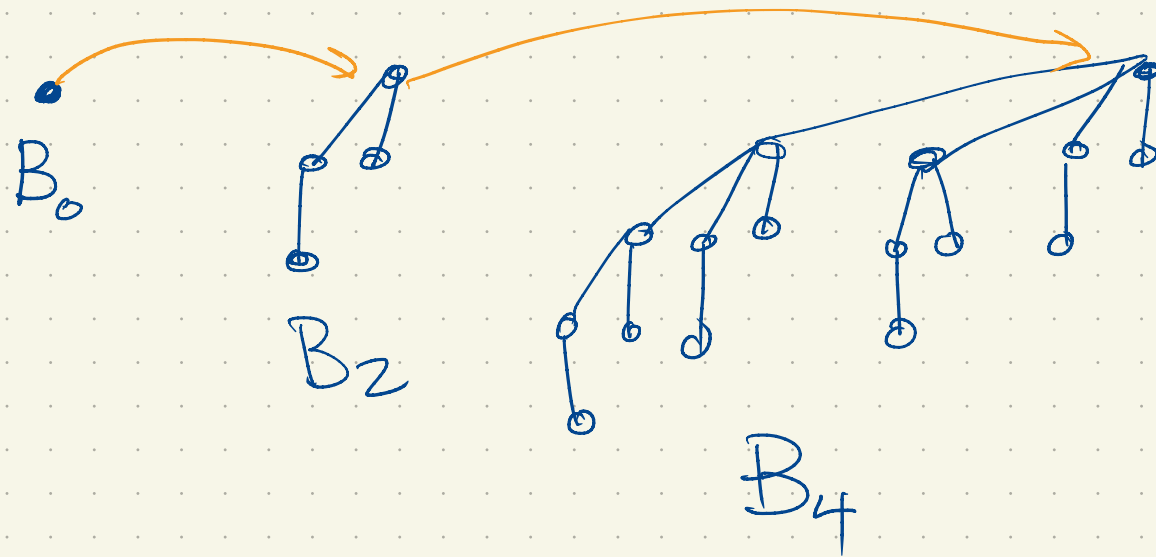
one B_2

⋮

one B_i (some i)

Index via a linked list sorted by
degree $0, \dots, i$

Ex List in orange trees in blue



(note: no B_1 or B_3 in this example)

length of list (in terms of # of nodes):

How to search:

Look at the roots & take min

Runtime:

But: can keep global ptr to minimum
(& just need to update it as you go)

↳ Runtime:

Next time:

Union $\rightarrow$ the whole point!